

PDF 1 - Assignment Decoder & Winning Strategy

Exactly what both tabs demand, what to build, and how to maximize the rubric in two days

CipherSchools Hiring Assignment - LLD Practice Platform | September 2026

The assignment in one sentence

Build a focused learner practice loop for Low-Level Design: choose a problem -> design -> submit -> receive explainable feedback -> review history -> try again. The reviewers explicitly prefer a small, working, extensible domain design over a large LMS or over-engineered infrastructure.

1. Non-negotiable requirements from Tab 1

Area	What must be demonstrated	Implementation evidence
Problem	Small set of LLD problems with clear requirements/context	Seed 4 problems: Parking Lot, Vending Machine, Elevator, Library
Practice	Learner starts an attempt and works on a solution	Structured text design editor with assumptions, classes, relationships, trade-offs
Submission	Submit and see status	Attempt state + immutable submission snapshot + visible status badge
Feedback	Useful design feedback; AI may be used	Fixed rubric, per-criterion score/evidence/concern/suggestion/confidence
History	Previous attempts support improvement	Attempt timeline + score deltas + recurring weaknesses
Core design	Clear classes/interfaces/responsibilities	Domain layer with Problem, Attempt, Submission, Evaluation, Rubric, Evaluator interfaces

2. What the reviewers are really testing

- Can you understand the learner problem before choosing technology?
- Can you model responsibilities and variation points cleanly at code level?
- Can you avoid assuming one reference design is the only correct LLD answer?
- Can you separate deterministic validation from judgment-heavy AI evaluation?
- Can the same practice flow later accept another submission format or evaluator without being rewritten?
- Can you make evaluation failure safe: save first, evaluate second, show state, allow retry, prevent duplicates?

3. Evaluation rubric translated into engineering priorities

Weight	What earns marks	Where to show it
25% LLD/domain design	Clean responsibilities, interfaces, behavior, extensibility	Design note + domain folder + tests
15% research	A few real tools, their workflows, gaps, and your direction	RESEARCH.md / research note
15% product thinking	A tight practice loop and useful retry/history behavior	UI flow + history comparison
15% feedback approach	Evidence-based rubric, deterministic + AI split	Evaluator module + structured result
10% extensibility	SubmissionFormat and Evaluator variation points	Interfaces + change-test explanation
10% implementation	Readable TypeScript, validation, loading/error states	Repository quality
5% tests	Important behavior + failure/edge cases	Vitest / Playwright
5% AI usage	3-5 meaningful accepted/rejected AI decisions	AI_USAGE.md

4. Scope: what NOT to build

- No Kubernetes, sharding, multi-region, service mesh, CDN architecture or microservice diagram.
- No full LMS, courses, certificates, social network, leaderboards, payments, complex admin portal.
- No support for text + code + UML + image uploads all at once. Pick one strong submission model.
- No arbitrary 0-100 LLM score without evidence.
- No design patterns added only to display pattern knowledge.

5. Recommended MVP

AI provider decision: use Google Gemini only for the MVP, called server-side with a Google AI Studio API key. Do not integrate OpenAI. Keep the Evaluator interface provider-neutral so another evaluator can be added later without changing the learner practice flow.

Feature	Decision	Why
Problems	4 seeded LLD problems	Enough variety; still focused
Submission format	Structured text design	Fast, expressive, avoids code execution complexity
Editor	Tabbed structured form: assumptions, classes, relationships, flows, trade-offs	Provides enough evidence for meaningful LLD judgment
Evaluation	Deterministic validation + AI rubric evaluator	Matches guide directly
History	Attempts list + compare latest vs previous	Makes learning loop visible
Auth	Demo account or lightweight sign-in	History needs ownership; keep simple
Architecture	TypeScript modular monolith	Assignment explicitly accepts monolith

6. 48-hour execution plan

Time	Goal
------	------

Hour 0-2	Research 3-4 tools; write learner problem, gaps, product direction
Hour 2-4	Freeze MVP, schema, domain interfaces, rubric and screen flow
Hour 4-10	Build data model, seeded problems, attempt/submission lifecycle
Hour 10-16	Build practice UI, autosave, submit state, history
Hour 16-21	Add evaluator + structured AI output + failure/retry path
Hour 21-27	Feedback UI + attempt comparison + accessibility/performance pass
Hour 27-32	Domain/unit tests + API edge cases + one E2E happy path
Hour 32-38	README, DESIGN.md, RESEARCH.md, AI_USAGE.md
Hour 38-44	Bug/security/performance pass, deployment
Hour 44-48	Fresh clone rehearsal, demo script, final submission

7. Small real-research note you can adapt

Research direction (outside the assignment source): Educative and DesignGurus emphasize repeated real-world OOD/LLD case studies, requirements, UML and implementation; LLDCanvas emphasizes a large problem library, staged hints and community comparison; the open-source LLD Arena combines an editor, hidden tests/rubrics, UML and optional AI grading. The gap your MVP can target is not "more content" but a short submit-feedback-retry loop with evidence-based feedback and attempt history.

Suggested sources: Educative OOD/LLD course overview; DesignGurus Grokking OOD; LLDCanvas; GitHub lld-arena. Accessed September 2026.

8. Definition of done

- Fresh user can open a problem, start an attempt, provide a meaningful design, submit once, and see Submitted/Evaluating/Completed or Failed.
- Feedback contains criterion, score, evidence from the submission, concern, suggestion and confidence.
- A failed evaluator does not lose the submission and can be retried safely.
- History shows at least two attempts and makes improvement visible.
- README runs from a clean clone; seeded demo data exists; tests pass; AI_USAGE.md explains real judgment.